

DAYANANDA SAGAR COLLEGE OF ENGINEERING

Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore-560078

Department of Artificial Intelligence and Machine Learning



INTERNSHIP REPORT

ON

“Automatic Scene and Emotion Detection with Timestamps in videos”

Submitted in partial fulfillment for the award of degree(21INT82)

BACHELOR OF ENGINEERING IN

Department of Artificial Intelligence and Machine Learning

Submitted by:

Anirudh Hegde (1DS21AI003)

Conducted at



Entertainment Technologists

Department of Artificial Intelligence and Machine Learning

Dayananda Sagar College of Engineering Bangalore-78

DAYANANDA SAGAR COLLEGE OF ENGINEERING

Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore-560078

Department of Artificial Intelligence and Machine Learning



CERTIFICATE

This is to certify that the Internship titled “Automatic Scene and Emotion Detection with Timestamps in videos” carried out by **Anirudh [1DS21AI003]**, a bonafide student of Dayananda Sagar College of Engineering, in partial fulfillment for the award of **Bachelor of Engineering**, in **Artificial Intelligence and Machine Learning** during the year 2024-2025. It is certified that all corrections/suggestions indicated have been incorporated in the report.

The internship report has been approved as it satisfies the academic requirements in respect course Internship (21INT82)

Signature of Reporting Head

Mr. Sriram Kumar Govindaraj

VP-Head of Product
Engineering
Entertainment
Technologists

Signature of Internship Coordinator

Prof. Kavya D N
Dept. of AIML

DSCE

Signature of HoD

Dr. Vindhya P Malagi

Dept. of AI & ML
DSCE

External Viva:

Name of the Examiner

1) _____

2) _____

Signature with Date

DAYANANDA SAGAR COLLEGE OF ENGINEERING

Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore-560078

Department of Artificial Intelligence and Machine Learning



D E C L A R A T I O N

I, Anirudh Hegde, final year student of Artificial Intelligence and Machine Learning, Dayananda Sagar College of Engineering, Bangalore - 560078, declare that the Internship has been successfully completed, in. This report is submitted in partial fulfillment of the requirements for the award of Bachelor's degree in Artificial Intelligence and Machine Learning, during the academic year 2024- 2025.

Date: 12/5/2025

USN: 1DS21AI003

Place: Bangalore

NAME: Anirudh Hegde

OFFER LETTER



Date: 29th January 2025
Address: 2555 Boulder Rd, Altadena, CA - 91001

Dear Anirudh,

Subject: **Offer of Internship at Entertainment Technologists Development Corp**

We are pleased to offer you an internship opportunity at Entertainment Technologists Development Corp. We are excited to welcome you as a valuable addition to our team and provide you with a valuable learning experience. This offer letter outlines the details of your internship and the terms and conditions associated with it.

1. You have been selected for the position of AI and ML Development Intern. The internship will commence on 1st February 2025 and is expected to end on 15th May 2025. You will be working for a minimum of 15 hours to maximum of 20 hours per week in this internship program. The internship will be remote. Please note that the internship may be subject to a probationary period, during which either party may terminate the agreement with a notice period of 2 weeks.
2. As an intern at Entertainment Technologists Development Corp, you will be working on an unpaid internship. The internship program is designed to give you learning experience in the media domain and related technologies. Entertainment Technologists Development Corp receives no direct financial benefit from this training program. You are willing accept to work in the unpaid internship offered by Entertainment Technologists Development Corp and will not claim for stipend for this internship program



offered by Entertainment Technologists Development Corp during and after the internship program.

3. Internship Responsibilities: During your internship, you will be expected to perform the following tasks and responsibilities:
 - Study of products, frameworks and API libraries
 - Develop and Test code snippets
 - Create weekly reports on the work assigned in the week
 - Submit a final report at the end of internship summarizing the work completed and articulating the learning experience in the internship.

You will receive guidance and feedback throughout the internship period.

4. Confidentiality and Intellectual Property: As an intern at Entertainment Technologists Development Corp, you will be exposed to confidential information and intellectual property. You are required to maintain the confidentiality of all proprietary information and trade secrets you come across during your internship. This includes but is not limited to client data, business strategies, internal processes, and any other information that is not publicly available. Additionally, any work, inventions, or creations developed during your internship shall be considered the property of Entertainment Technologists Development Corp, and you shall not claim any rights or ownership over them.
5. Code of Conduct: As an intern, you are expected to adhere to the company's code of conduct and maintain a high standard of professionalism and integrity. This includes, but is not limited to, being punctual, respecting colleagues, following company policies and



procedures, and conducting yourself in a manner that positively represents Entertainment Technologists Development Corp.

6. Termination: Either party may terminate this internship agreement with a notice period of 2 weeks in writing. Entertainment Technologists Development Corp reserves the right to terminate the internship at any time for reasons such as poor performance, violation of company policies, or any other valid reason.
7. Acceptance: To accept this internship offer, please sign and return a copy of this offer letter by **30th January 2025**.
8. You will be responsible to get the required work authorization approvals to be eligible for working as an intern from the requisite authorities. Entertainment Technologists Development Corp will not be providing you any assistance on work authorization, will not be responsible to attain work authorization and assumes that you will have the requisite approvals to work as intern in this internship program.

We are excited to have you join our team as an intern and look forward to a mutually beneficial experience.

Once again, congratulations on your selection for this internship opportunity. We are confident that your skills and enthusiasm will make a positive impact at Entertainment Technologists Development Corp.

Yours sincerely,
Uday Ghare
Co-Founder



Entertainment Technologists Development Corp

Please sign below to accept this offer:

Name: Anirudh Hegde

Date: 29/01/2025


Signature

COMPLETION CERTIFICATE

ACKNOWLEDGEMENT

This Internship is a result of accumulated guidance, direction and support of several important persons. We take this opportunity to express our gratitude to all who have helped us to complete the Internship.

We express our sincere thanks to our Principal **Dr. B G Prasad**, for providing us adequate facilities to undertake this Internship.

We would like to thank **Dr. Vindhya P Malagi**, Head of Department – AI & ML, for providing us an opportunity to carry out Internship and for her valuable guidance and support.

We would like to thank **Mrs Sukhada Joshi**, for guiding me during the period of internship.

We express our deep and profound gratitude to our Internship Co-ordinator, **Kavya D N**, Assistant Professor – AI & ML, for her keen interest and encouragement at every step in completing the Internship.

I would also like to extend my thanks to **Prof. Ensteih Silvia**, Internal guide, for her consistent guidance, timely feedback, and unwavering support throughout the internship. Her mentorship helped me stay aligned with both academic and industry expectations.

We would like to thank all the faculty members of AI & ML department for the support extended during the course of Internship.

We would like to thank the non-teaching members of the Department of AI & ML, for helping us during the Internship.

Last but not the least, we would like to thank our parents and friends without whose constant help, the completion of Internship would have not been possible.

Anirudh Hegde [1DS21AI003]

TABLE OF CONTENTS

Sl no	Description	Page no
1	INTRODUCTION	1
2	DESCRIPTION OF ORGANIZATION	5
3	SCOPE OF WORK	8
4	LEARNING OBJECTIVES AS GIVEN BY COMPANY	10
5	CHALLENGES AND SOLUTIONS	11
6	ACHIEVEMENTS AND CONTRIBUTION	13
7	REFLECTION AND ANALYSIS	34
8	RECOMMENDATIONS	42
	CONCLUSION	43
	REFERENCES	44

CHAPTER 1

INTRODUCTION

1.1 Introduction

Focuses on detecting scene transitions in videos and extracting their corresponding timestamps using Python. It leverages video processing libraries such as OpenCV and PySceneDetect to automatically identify shot boundaries, enabling users to segment videos based on visual changes. The output includes a list of scenes along with their start and end times, which is useful for video summarization, content indexing, and editing workflows.

OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning software library. OpenCV provides a common infrastructure for computer vision applications and accelerates the use of machine perception in commercial products. It includes numerous algorithms for image processing, object detection, facial recognition, and more.

Some key advantages of equipment performance prediction are:

1. **Automated Scene Segmentation:** Eliminates the need for manual video scanning by accurately detecting scene changes.
2. **Lightweight & Efficient:** Optimized for performance using libraries like PySceneDetect and OpenCV, ensuring fast processing even on standard hardware.
3. **Precise Timestamps:** Provides exact start and end times for each scene, making it ideal for editing and indexing.
4. **Customizable Thresholds:** Scene detection sensitivity can be tweaked to suit different video styles and content types.

Simple Scene Boundary/Shot Transition Detection with OpenCV

A “scene boundary” or a “shot transition” in a movie, TV show, or video is a natural way for the producers and editors to indicate that the current scene is complete and the next scene is starting.

The most common type of scene boundary is a “fade to black”. Just as the name suggests, this is when a scene ends and the video fades to black, then fades back in, indicating that the next scene is starting. Using computer vision, we seek to automatically find these scene boundaries, enabling us to create a “smart video segmentation” system.

Detection algorithms are created by implementing the SceneDetector interface. Detectors are typically attached to a SceneManager when processing videos, however they can also be used to process frames directly.

AdaptiveDetector compares the difference in content between adjacent frames similar to ContentDetector except the threshold isn’t fixed, but is a rolling average of adjacent frame changes. This can help mitigate false detections in situations such as fast camera motions.

ContentDetector compares the difference in content between adjacent frames against a set threshold/score, which if exceeded, triggers a scene cut.

Video transition detection also known as scene cut detection is an important video processing task that aims at finding interjoined scenes to identify visual content. It's a fundamental step for video post-production tasks that may involve the addition of newer scenes to visual content. For example, the best duration to insert advertisements into a video streaming service is between the video scene transitions to reduce the prediction of visual changes. Additionally, a content-based video retrieval process requires efficient access to a huge amount of video frame archives which is solved by selecting a representative picture from each scene.

Video frame indexing and searching: Locates specific frames inside a video by taking keywords or queries/questions in a natural language. For example, a query input such as show me all films where there's a scene for people watching TV traditionally requires the slicing and storage of all video frames in a database. However, this is inefficient when working with multiple videos with a long video length. The scene detection and cut function could store, and process only the representative frames which reduces the searching space. Based on the example in this article, the total number of frames can reduce from 1440 to 44 indicating a 97% reduction in the processing time.

Objectionable video frame detection: Checks if a newly uploaded video contains objectionable scenes that are suitable for some audiences. Similarly, it's inefficient to check all the frames from all the videos on a large scale and scene detection could significantly improve its efficiency.

Best places to insert ads: Finds the video transition durations for advertisement slots so that viewer experiences are less interrupted. This task explicitly requires scene detection as a dependency.

The content-aware scene detector (detect-content) detects jump cuts in the input video. This is typically what people think of as "cuts" between scenes in a movie - given two adjacent frames, do they belong to the same scene? The content-aware scene detector finds areas where the difference between two subsequent frames exceeds the threshold value that is set (a good value to start with is --threshold 27).

Internally, this detector functions by converting the color space of each decoded frame from RGB into HSV. It then takes the average difference across all channels (or optionally just the value channel) from frame to frame. When this exceeds a set threshold, a scene change is triggered.

PySceneDetect implements a variety of different detection algorithms which can be used independently or combined depending on the source material being analyzed.

- **Adaptive content scene detection (detect-adaptive):** uses rolling average of differences in HSL colorspace combined with thresholding to detect shot changes (fast cut)
- **Content-aware scene detection (detect-content):** uses differences in HSL colorspace combined with filtering to detect shot changes (fast cut)
- **Content-aware scene detection (detect-hash):** uses perceptual hashing to determine differences between frames to find shot changes (fast cut)
- **Content-aware scene detection (detect-hist):** uses differences in histograms of Y channel of frames after conversion to YUV (fast cut)
- **Threshold scene detection (detect-threshold):** uses average frame intensity (brightness) to detect slow transitions (fade in/out)

Ffmpeg is a universal media converter. It can read a wide variety of inputs - including live grabbing/recording devices - filter, and transcode them into a plethora of output formats. ffmpeg reads from an arbitrary number of inputs (which can be regular files, pipes, network streams, grabbing devices, etc.), specified by the -i option, and writes to an arbitrary number of outputs, which are specified by a plain output url. Anything found on the command line which cannot be interpreted as an option is an output url.

Each input or output can, in principle, contain any number of elementary streams of different types (video/audio/subtitle/attachment/data), though the allowed stream counts and/or types may be limited by the container format. Selecting which streams from which inputs will go into which output is either done automatically or with the -map option.

In conclusion, Scene Detection with Timestamps in Videos demonstrates an effective and automated approach to scene detection in videos using Python. By leveraging powerful libraries like OpenCV and PySceneDetect, it simplifies the process of identifying visual transition.

CHAPTER 2

DESCRIPTION OF ORGANIZATION



2.1 Introduction

Ent Tech Dev Corp., a leading software development and consulting firm, has been at the forefront of technological innovation since 2023. With over 28 years of experience, we have mastered the art of creating advanced software solutions that are tailored to the unique needs of our customers. Our expertise lies in our ability to understand the complexities of your business and develop software solutions that streamline operations, enhance customer engagement, and drive growth.

As a minority-owned business, we take pride in our diverse team of experts who bring fresh perspectives and innovative ideas to the table. Our commitment to excellent customer service is evident in our quick response time - we guarantee a response within 24 hours. We also offer free consultations, providing potential clients with an opportunity to discuss their needs and understand how our solutions can help them achieve their business goals.

Choosing Ent Tech Dev Corp. means partnering with a seasoned software development company that understands the unique challenges and opportunities in doing business. Our extensive experience, coupled with our commitment to customer service, ensures that we deliver solutions that not only meet but exceed your expectations. We are not just a service provider; we are a partner who is invested in your success. Our solutions are designed to help you stay ahead of the curve in a rapidly evolving industry.

2.2 Founders

Uday Ghare – CEO & Co-Founder



As the former Vice President of delivery for North America for TechMahindra, Uday has delivered massively complex software projects for some of the largest media and telecommunication companies in the world.

Uday has over 25 years of experience in building software teams around the globe including sourcing staff, training, outsourcing, and managing their growth. Uday is adept at translating sometimes disparate customer requirements into detailed development proposals and executing against those plans with continuous creative approaches to problem-solving. At TechMahindra, Uday founded the M&E Practice and delivered dozens of projects for some of the largest media companies from greenlight to consumer streaming apps and the monetization, CRM, CMS, and infrastructure to support them.

Mark Turner - President

Mark is a creative strategist and technologist with a passion for finding innovative solutions to tough business issues. As the President of Entertainment Technologists Development Corp., Mark leverages his nearly 30 years of management experience helping clients from Fortune 500 companies through to innovative and fast-moving start-ups.

As a former video game developer, Mark has a thorough technical background but with a rare combination of management experience across product development, business development, corporate strategy, marketing, and finance. Mark has had senior roles Ryff, Technicolor, Dolby, Microsoft, BT, and Eutelsat and is always innovating and learning. He was the founding Chairman of the Consumer Technology Associations AR/VR Committee, a founding board member of the UHD Alliance and served on various leadership boards at DEG, UltraViolet, and the Interactive TV Alliance.

Most Mark wrote and published the 2030 Vision for the Future of media production, VFX, and post in the cloud on behalf of all the major Hollywood studios, a detailed guide to the business/workflow impacts of Virtual Production and an interactive calculator for production costs. Mark has incubated and launched several entrepreneurial ventures is writing a book and producing two TV shows.

CHAPTER 3

SCOPE OF WORK

During the internship tenure, I was involved in diverse projects encompassing Scene Detection from the videos along with timestamps. Also retrieving emotions from video or movie clips and creating new video with emotion-filled scenes.

The primary objective of this project is to develop a system capable of detecting human emotions from video input and associating these emotional states with accurate timestamps. This solution aims to facilitate emotion-aware analytics for applications such as user experience research, content moderation, behavioral analysis, and interactive media systems.

1. Data Preprocessing:

Use sample videos of varying length and content for testing, preferably with diverse demographics. Face detection using models like MTCNN, OpenCV Haar Cascades, or Dlib.

2. Emotion Recognition:

- Emotion classification using deep learning models (e.g., FER+, VGGFace, ResNet, MobileNet).
- Optionally overlay predictions on video and export annotated video.

3. Timestamp Mapping

- Associate each emotion prediction with its corresponding timestamp
- Handle variable frame rates and ensure temporal alignment

4. Output Generation:

- Store results in JSON/CSV format with fields: [timestamp, emotion, confidence, frame_id].
- Optionally overlay predictions on video and export annotated video.

The development of an emotion detection system with timestamped video analysis presents a powerful tool for understanding human emotional behavior in dynamic contexts. By leveraging state-of-the-art facial recognition and deep learning models, the system effectively identifies emotions such as happiness, sadness, anger, and more, with each detection precisely mapped to its corresponding time within the video.

This approach enables the extraction of temporal emotional patterns, which are valuable in domains like content analysis, human-computer interaction, surveillance, education, and therapy. The methodology adopted—ranging from frame extraction to model-based classification and timestamp logging—proved both robust and scalable.

Although current limitations include the exclusion of audio cues and complex multi-person tracking, the foundational pipeline is extensible for future enhancements. Overall, the project demonstrates the feasibility and utility of emotion-timestamped video analysis, laying the groundwork for real-time applications and multi-modal emotional intelligence systems.

This lays a solid foundation for advanced video analytics applications across diverse fields.

CHAPTER 4

LEARNING OBJECTIVES

1. Understand the Core Concepts of Video Scene Detection

Video scene detection, also known as scene segmentation or scene boundary detection, is the process of automatically identifying and segmenting different scenes within a video. This is crucial for various applications such as video summarization, content-based video Retrieval.

2. Gain Hands-On Experience Using PySceneDetect and OpenCV

Install, configure, and use these libraries to work with real video files. This improves your skills in leveraging open-source Python tools for visual data processing. It also enhances your confidence in navigating video frame extraction. The hands-on experience will help in projects requiring custom video analysis solutions.

3. Learn to Manipulate and Process Video Frames Using Python

Teaches the practical use of OpenCV functions for frame extraction. Understanding how image matrices are used to compare frames. This helps in developing both low-level and high-level understanding of video data.

4. Explore Techniques to Optimize Detection Thresholds

Learn to tune parameters to detect more precise cuts. Experimented with values that impact scene change sensitivity. Optimization is key to balancing false positives and missed detections. These skills transfer well into other ML or CV parameter tuning tasks.

5. Structure Output Data Efficiently for Downstream Applications

Lastly, the internship provided me with an opportunity to observe and organize timestamp data in readable formats like CSV or TXT. This teaches how to prepare data for media workflows or content-based retrieval. Gained experience in aligning video content with structured metadata.

CHAPTER 5

CHALLENGES AND SOLUTIONS

1. Detecting Gradual Transitions (Fades, Dissolves)

Traditional scene detection algorithms work best for hard cuts, but struggle with gradual transitions. These transitions often have subtle pixel changes that can be missed by threshold-based detection. To address this, sensitivity tuning was applied and alternative detection modes in PySceneDetect were explored to improve accuracy for fade-ins and fade-outs.

2. Handling Low-Contrast Scenes

Videos with similar lighting or color tones between scenes posed challenges in distinguishing scene boundaries. This sometimes led to missed detections or false positives. To solve this, histogram comparison methods and pixel intensity difference strategies were experimented with to better capture subtle variations.

3. Processing High-Resolution Videos Efficiently

High-definition videos increase computational load, resulting in slower frame-by-frame analysis. To mitigate this, the videos were downsampled to a lower resolution before processing without significant loss of accuracy. Batch processing techniques and frame skipping strategies were also employed to speed up execution.

4. Emotion Detection Accuracy

Detecting emotions from frames was challenging due to poor lighting, low face resolution, or occluded faces. Preprocessing steps like face alignment, brightness normalization, and confidence thresholding were added to improve emotion recognition reliability. Only faces with high detection confidence were analyzed.

5. Balancing Speed and Accuracy

Fine-tuning the threshold for scene changes directly impacted processing time and detection quality. Lower thresholds increased detection sensitivity but also raised false alarms. The solution was to find an optimal balance via trial-and-error, benchmarking multiple video types, and integrating a feedback-based refinement loop.

HARDWARE AND SOFTWARE REQUIREMENTS

Software Requirements:

Operating System: Windows, macOS, or Linux

Python 3.x installed with pip

Libraries: OpenCV, PySceneDetect, NumPy, Pandas, FFmpeg (optional), DeepFace/FER (optional for emotion detection)

Code Editor: VSCode, PyCharm, or Jupyter Notebook

Git (for version control)

Hardware Requirements:

CPU: Multi-core processor (Intel i5/Ryzen 5 or higher recommended)

RAM: Minimum 8 GB (16 GB recommended for processing high-resolution videos)

Storage: At least 1 GB of free disk space (more for storing video files and outputs)

GPU (optional): For accelerating emotion detection if using deep learning models

Tools & Technologies Used:

- i. Python 3.x
- ii. OpenCV
- iii. PySceneDetect
- iv. FFmpeg (optional)

CHAPTER 6

ACHIEVEMENTS AND CONTRIBUTION

During my internship at Entertainment Technologists, I had the opportunity to contribute significantly to the Scene Detection in long videos or movie clips along emotion detection by utilizing FER and Computer Vision library for the project. However it enabled me to strengthen my Video Segmentation, Computer Vision and Retrieval skills.



1. Emotion Recognition Integration

Implemented the FER library to extract and classify facial emotions from video frames, enabling deeper scene understanding. This added an emotional layer to the video analysis, allowing for richer content interpretation. Emotion recognition integration involves incorporating facial expression analysis into an application using tools like OpenCV for face detection and libraries such as FER or deep learning models for emotion classification. The process begins with real-time video input or static images, where OpenCV identifies and extracts facial regions. These facial areas are then passed to an emotion recognition model—such as one provided by the FER library or a custom-trained CNN—which analyzes facial features and predicts the most likely emotion (e.g., happy, sad, angry, surprised). The results can be used to personalize user experiences, trigger automated responses, or analyze behavioral patterns. Emotion recognition can be integrated into a wide range of applications, including virtual assistants, surveillance systems, mental health platforms, and interactive media, providing a deeper understanding of human behavior and emotional context in real time.

2. Scene Segmentation with OpenCV

Utilized OpenCV to process video frame-by-frame and detect hard cuts based on pixel differences. This automated scene segmentation significantly reduced manual editing efforts. Scene segmentation using OpenCV and the FER library involves detecting and analyzing individual frames from a video or stream to identify emotional expressions within distinct scenes. OpenCV is first used to process the video frame-by-frame, applying face detection algorithms to locate faces within each frame. Once a face is detected, the FER library analyzes the facial features and classifies the emotion, such as happy, sad, angry, or surprised. These emotional cues are then used as a basis for scene segmentation—grouping sequences of frames that share similar emotional content. This allows for the extraction of emotionally coherent scenes, enabling applications such as highlight generation, sentiment-aware video summarization, and emotion-driven content indexing. The combination of OpenCV for visual processing and FER for emotion classification creates a lightweight and effective pipeline for emotion-based scene segmentation.

3. Timestamp Generation

Automatically generated accurate start and end timestamps for each detected scene. This structured output facilitated easier navigation, editing, and indexing of video content. Timestamp generation using the FER library involves analyzing a video frame-by-frame to detect emotions and record the exact time (timestamp) when each recognized emotion occurs. The process begins with loading a video and using OpenCV to iterate through its frames. For each frame (or at specific intervals for performance), the FER library detects faces and classifies the expressed emotion. When a significant emotion is detected—based on intensity or change—a timestamp is recorded, correlating to the frame's position in the video timeline. This enables the creation of an emotion timeline, where segments of the video are tagged with emotions like “happy at 00:02:14” or “sad at 00:07:31.” This approach is valuable for applications such as emotional scene segmentation, content moderation, user behavior analysis, and automated video summarization.

4. Data Export & Structuring

Designed and implemented output formatting in readable file types like PNGs and JPGs. These structured exports supported downstream applications like content retrieval and machine learning labeling. Data export and structuring in emotion recognition involves capturing the detected emotions, their corresponding timestamps, and additional metadata (such as confidence scores or face count) into a well-organized format. Once the FER library processes a video frame and identifies emotions, the results—typically including the timestamp, detected emotion, and confidence level—are stored in a structured dataset. This data is then exported to formats like PNGS or JPGs for spreadsheet analysis, JSON for web integration, or a database for persistent storage. Structuring the data allows for easy querying, filtering by emotion type or time range, and visualization (e.g., emotion over time graphs). This step is essential for tasks like behavioral analytics, user sentiment reporting, or training more complex models using labeled emotional data.

5. Pipeline Optimization

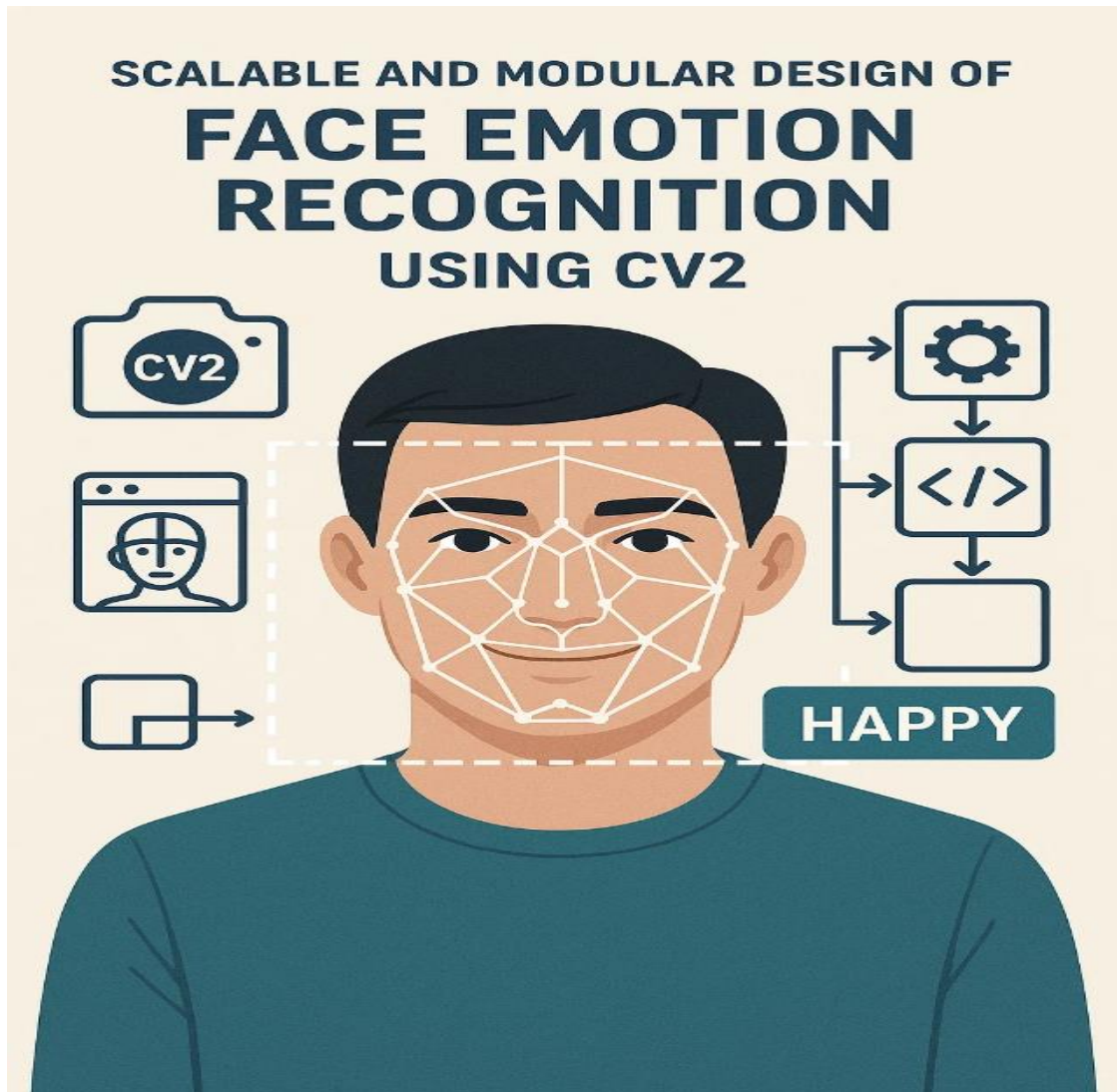
Optimized the video processing pipeline to handle diverse video formats and resolutions. This ensured efficient performance even on mid-range hardware setups. Pipeline optimization in an emotion recognition system involves enhancing each stage of the process—from face detection to emotion classification and data export—to improve speed, accuracy, and resource usage. Techniques include downsampling frames (e.g., analyzing every nth frame instead of all), using GPU-accelerated libraries like OpenCV with CUDA, to reduce inference time. Batch processing frames, avoiding redundant detections by tracking faces across frames, and using multithreading or asynchronous I/O also contribute to smoother performance. Additionally, selectively exporting only significant emotional changes, rather than logging every frame, helps in reducing data volume and improving clarity. A well-optimized pipeline ensures real-time responsiveness and scalability, making it suitable for production-level applications like surveillance, UX analytics, or interactive systems.

6. Combined Scene and Emotion Analysis

Merged both scene transition detection and emotion recognition to generate a dual-layered analysis. This provided a more holistic view of the video content. Combined scene and emotion analysis merges traditional scene segmentation with facial emotion recognition to understand not just what is happening in a video, but how people feel during each scene. Using OpenCV or deep learning-based scene detectors, the video is divided into distinct segments based on visual or temporal changes (like shot boundaries or content shifts). Simultaneously, the FER library is applied to detect faces and classify emotions within each segment. This dual-layer analysis enables tagging scenes with both contextual metadata (e.g., indoor, outdoor, crowd) and emotional tone (e.g., joyful, tense, surprised). The result is a detailed emotional map of the video, useful for applications like highlight generation, sentiment-driven video summarization, content moderation, advertising, and audience engagement analytics. Structuring this data with timestamps, dominant emotion per scene, and scene descriptors enables powerful downstream tasks such as emotion-based retrieval, recommendation, and storytelling.

7. Scalable & Modular Design

Engineered the codebase to be modular and extensible, allowing for easy integration of advanced features like emotion trend graphs, thumbnails, or scene classification models. The image illustrates a scalable and modular workflow for face emotion recognition using OpenCV (CV2). The process begins with capturing facial input through a camera, where OpenCV detects and isolates the face in real time. Once the face is identified, facial landmarks such as eyes, eyebrows, nose, and mouth are extracted using either built-in OpenCV tools or external libraries like FER or CV2. These landmarks are then processed in a modular pipeline, ensuring that each step—detection, feature extraction, and classification—can be independently scaled or improved. The extracted features are passed into a machine learning or deep learning model trained to recognize various emotional states. Finally, the model classifies the emotion based on facial expression, as indicated by the “HAPPY” or “SAD” label in the image. This modular and scalable design allows for flexible integration and efficient performance in real-world emotion recognition systems.



The image illustrates a scalable and modular workflow for face emotion recognition using OpenCV (CV2). The process begins with capturing facial input through a camera, where OpenCV detects and isolates the face in real time. Once the face is identified, facial landmarks such as eyes, eyebrows, nose, and mouth are extracted using either built-in OpenCV tools or external libraries like FER or CV2. These landmarks are then processed in a modular pipeline, ensuring that each step—detection, feature extraction, and classification—can be independently scaled or improved. The extracted features are passed into a machine learning or deep learning model trained to recognize various emotional states. Finally, the model classifies the emotion based on facial expression, as indicated by the “HAPPY” label in the image. This modular and scalable design allows for flexible integration and efficient performance in real-world emotion recognition systems.

FER (Facial Expression Recognition):

Facial Expression Recognition (FER) is the task of identifying emotions or expressions from human faces, a crucial component in applications like user sentiment analysis, security surveillance, customer feedback systems, and interactive AI-based assistants. The primary focus is on detecting emotions such as happiness, sadness, anger, surprise, disgust, and fear from images or video frames.

FER models are generally based on deep learning techniques, specifically convolutional neural networks (CNNs). However, their accuracy can drop under challenging conditions like varying lighting, occlusion (e.g., glasses or hair covering part of the face), or unusual facial expressions. Pre-trained models, such as those from libraries like FER in Python, make the deployment of FER easier, reducing the need for training large models from scratch.

PySceneDetect:

PySceneDetect is a Python library used for detecting scene boundaries (or cuts) in videos, crucial for video analysis and media processing tasks. It helps in dividing videos into scenes, allowing better indexing, summarization, or further analysis like action recognition, content tagging, or facial expression changes across scenes.

Here are some of the things people are using PySceneDetect for:

- splitting home videos or other source footage into individual scenes
- automated detection and removal of commercials from PVR-saved video sources
- processing and splitting surveillance camera footage
- statistical analysis of videos to find suitable "loops" for looping GIFs/cinemagraphs
- academic analysis of film and video (e.g. finding mean shot length)

The content-aware scene detector (detect-content) detects jump cuts in the input video. This is typically what people think of as "cuts" between scenes in a movie - given two adjacent frames, do they belong to the same scene? The content-aware scene detector finds areas where the difference between two subsequent frames exceeds the threshold value that is set (a good value to start with is --threshold 27).

The adaptive content detector (detect-adaptive) compares the difference in content between adjacent frames similar to detect-content but instead using a rolling average of adjacent frame changes. This helps mitigate false detections where there is fast camera motion.

Dissolve Detection:

Depending on the length of the dissolve and parameters being used, detection accuracy for these types of cuts can vary widely. A method to improve accuracy with minimal performance loss is an open problem.

Internally, this detector functions by converting the colorspace of each decoded frame from RGB into HSV. It then takes the average difference across all channels (or optionally just the value channel) from frame to frame. When this exceeds a set threshold, a scene change is triggered.

Detect-content also has edge detection, which can be enabled by providing a set of 4 numbers in the form (delta_hue, delta_sat, delta_lum, delta_edges). Changes in edges are typically larger than the other components, so threshold may need to be increased accordingly. For example, -w 1.0 0.5 1.0 0.2 -t 32 is a good starting point to use with edge detection. The default weights are --weights 1.0 1.0 1.0 0.0 which does not include edges, but this may change in the future.

```
import os

import cv2
from fer import FER

emotion_detector = FER(mtcnn=True)

directory = "/home/anirudh/detect-scene/movies-videos"
output_dir = "/movie_output_happy_videos"

os.makedirs(output_dir, exist_ok=True)
#
for name in os.listdir(directory):
    path_to_video = os.path.join(directory, name)

    if not os.path.isfile(path_to_video):
        continue

    print(f"Processing: {name}")

    # path_to_video = "/home/anirudh/detect-scene/videos/3idiots.mp4"
    cap = cv2.VideoCapture(path_to_video)
    fps = int(cap.get(cv2.CAP_PROP_FPS))
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH) * 0.5)
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT) * 0.5)

    output_path = os.path.join(output_dir, name + "_happy_video.mp4")

    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    out = cv2.VideoWriter(output_path, fourcc, fps // 3, (width, height))

    frame_count = 0
    happy_frames = 0
```

```

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    if frame_count % 3 == 0:
        frame = cv2.resize(frame, size=(width, height))
        result = emotion_detector.detect_emotions(frame)

        for face in result:
            emotions = face["emotions"]
            dominant_emotion = max(emotions, key=emotions.get)

            # if emotions["happy"] >= 0.40 or dominant_emotion == "happy":
            if emotions["happy"] >= 0.20:
                box = face["box"]
                cv2.rectangle(frame, (box[0], box[1]), (box[0] + box[2], box[1] + box[3]), (255, 0, 0),
                               2)
                cv2.putText(frame, text="HAPPY", org=(box[0], box[1] - 10), cv2.FONT_HERSHEY_SIMPLEX, fontScale=0.4, color=(255,
                                                                                               0, 0))
                out.write(frame)
                happy_frames += 1

        frame_count += 1

cap.release()
out.release()

print(f"Processed video saved as {output_path} with {happy_frames} 'happy' frames.")

```

This Python script processes a collection of video files to extract and save only the segments where people appear visibly happy. It begins by setting up the required directories for input videos and output results. Using the FER library with MTCNN face detection enabled the script analyzes each video frame to detect emotions. Videos are loaded with OpenCV, and every third frame is resized to reduce processing time. For each detected face in a frame, the script evaluates the "happy" emotion score. If the score is 20% or higher, the frame is annotated with a rectangle around the face and a "HAPPY" label, then written to a new output video. This helps generate highlight reels focusing solely on happy moments. Once all frames are processed, the video is saved with an informative summary of how many "happy" frames were captured. This approach is useful for emotional content curation, user engagement analysis, or creating personalized media experiences based on facial expressions.

When combining FER and PySceneDetect, you can create a powerful system for analyzing emotional content in videos by detecting when and how emotions evolve across different scenes. Here's how it can be applied:

Real-time Emotion Analysis Across Scenes: For a video, you could apply PySceneDetect to segment it into scenes and then use FER to analyze the emotions of subjects within each scene. This would allow a more granular understanding of how emotions evolve in different parts of a video (e.g., a character's mood changes in different scenes of a movie).

User Sentiment Analysis in Video Content: If analyzing user-generated content, such as customer reviews or tutorials, you could detect shifts in facial expressions (via FER) to understand how the user feels during different parts of the video (e.g., excitement in one scene, frustration in another).

PySceneDetect implements a variety of different detection algorithms which can be used independently or combined depending on the source material being analyzed.

- Adaptive content scene detection (detect-adaptive): uses rolling average of differences in HSL colorspace combined with thresholding to detect shot changes (fast cut)
- Content-aware scene detection (detect-content): uses differences in HSL colorspace combined with filtering to detect shot changes (fast cut)
- Content-aware scene detection (detect-hash): uses perceptual hashing to determine differences between frames to find shot changes (fast cut)
- Content-aware scene detection (detect-hist): uses differences in histograms of Y channel of frames after conversion to YUV (fast cut)
- Threshold scene detection (detect-threshold): uses average frame intensity (brightness) to detect slow transitions (fade in/out)

Ffmpeg is a universal media converter. It can read a wide variety of inputs - including live grabbing/recording devices - filter, and transcode them into a plethora of output formats.

Ffmpeg reads from an arbitrary number of inputs (which can be regular files, pipes, network streams, grabbing devices, etc.), specified by the `-i` option, and writes to an arbitrary number of outputs, which are specified by a plain output url. Anything found on the command line which cannot be interpreted as an option is considered to be an output url.

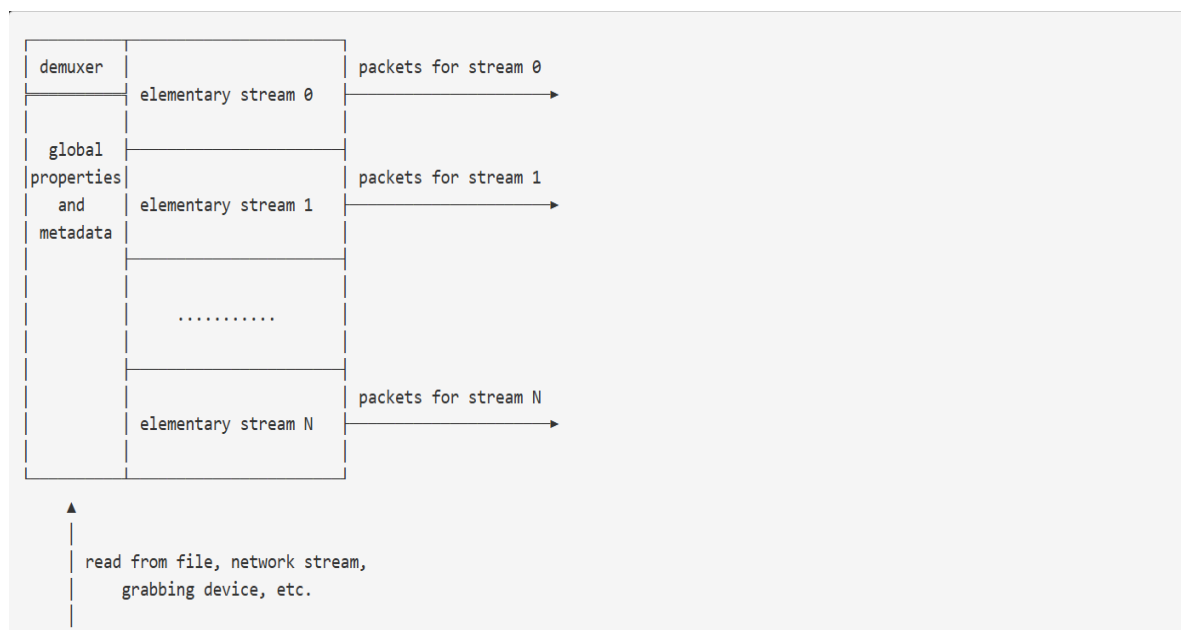
Ffmpeg builds a transcoding pipeline out of the components listed below. The program's operation then consists of input data chunks flowing from the sources down the pipes towards the sinks, while being transformed by the components they encounter along the way.

The following kinds of components are available:

Demuxers (short for "demultiplexers") read an input source in order to extract

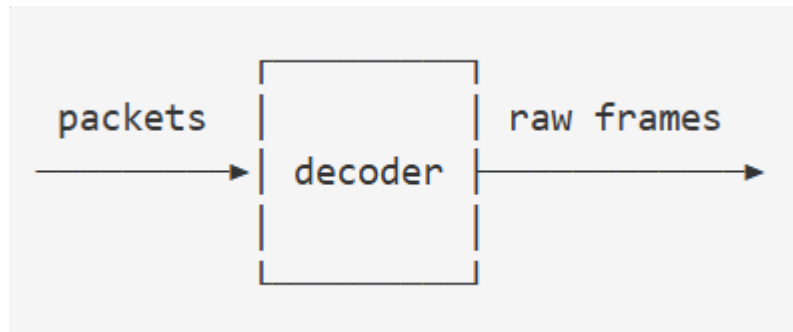
- global properties such as metadata or chapters;
- list of input elementary streams and their properties. One demuxer instance is created for each `-i` option, and sends encoded packets to decoders or muxers.

A schematic representation of a demuxer looks like this:



Decoders receive encoded (compressed) packets for an audio, video, or subtitle elementary stream, and decode them into raw frames (arrays of pixels for video, PCM for audio). A decoder is typically associated with (and receives its input from) an elementary stream in a demuxer, but sometimes may also exist on its own.

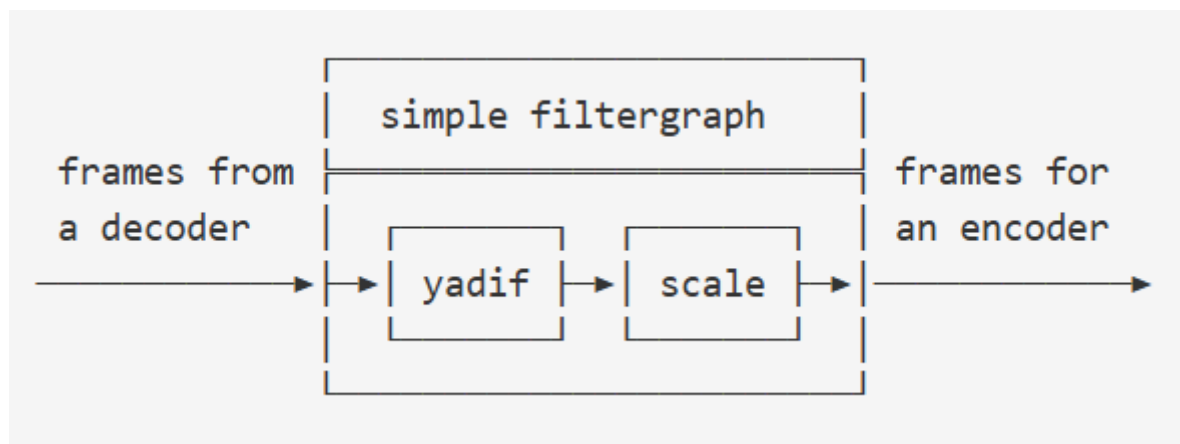
A schematic representation of a decoder looks like this:



Filtergraphs process and transform raw audio or video frames. A filtergraph consists of one or more individual filters linked into a graph. Filtergraphs come in two flavors - simple and complex, configured with the `-filter` and `-filter_complex` options, respectively.

A simple filtergraph is associated with an output elementary stream; it receives the input to be filtered from a decoder and sends filtered output to that output stream's encoder.

A simple video filtergraph that performs deinterlacing (using the `yadif` deinterlacer) followed by resizing (using the `scale` filter) can look like this:



In other literature, demuxers are sometimes called splitters, because their main function is splitting a file into elementary streams (though some files only contain one elementary stream).

A preset file contains a sequence of option=value pairs, one for each line, specifying a sequence of options which would be awkward to specify on the command line. Lines starting with the hash ('#') character are ignored and are used to provide comments.

Each input or output can, in principle, contain any number of elementary streams of different types (video/audio/subtitle/attachment/data), though the allowed stream counts and/or types may be limited by the container format. Selecting which streams from which inputs will go into which output is either done automatically or with the -map option





PySceneDetect employs both threshold-based and content-aware (e.g., machine learning or pixel-based) techniques to detect scene transitions, making it suitable for a range of videos from films to home videos. The content-aware method allows for better accuracy when detecting scene changes where visual content significantly shifts. On the other hand, simple threshold methods might work well in controlled settings (like recorded webinars or lectures) but may be less effective in more dynamic or creative video styles.

```
Analyzing video: /content/drive/mydrive/avengers.mp4  
INFO:pyscenedetect:Detecting scenes...
```



```
Detected 29 scenes:
```

```
Scene 1:  
Start Time: 0.00s  
End Time: 87.10s  
Duration: 87.10s
```

```
Scene 2:  
Start Time: 87.10s  
End Time: 118.57s  
Duration: 31.47s
```

```
Scene 3:  
Start Time: 118.57s  
End Time: 119.97s  
Duration: 1.40s
```

```
Scene 4:  
Start Time: 119.97s  
End Time: 157.50s  
Duration: 37.53s
```

```
Scene 5:  
Start Time: 157.50s  
End Time: 158.03s  
Duration: 0.53s
```

```
Scene 6:  
Start Time: 158.03s  
End Time: 158.63s  
Duration: 0.60s
```

```
Scene 7:  
Start Time: 158.63s  
End Time: 160.00s  
Duration: 1.37s
```

```
Scene 8:  
Start Time: 160.00s
```

```
Scene 1: 00:00:00.000 --> 00:00:12.420  
Scene 2: 00:00:12.421 --> 00:00:27.803  
Scene 3: 00:00:27.804 --> 00:00:49.117  
...
```

Timestamps retrieved

Emotion Analysis Results:

```

🔗 Scene: ./scenes/avengers_scene_000.mp4
  Face: ./faces/face_39_0.jpg → Emotion: neutral
  Face: ./faces/face_40_0.jpg → Emotion: neutral
  Face: ./faces/face_41_0.jpg → Emotion: neutral
  Face: ./faces/face_42_0.jpg → Emotion: neutral
  Face: ./faces/face_43_0.jpg → Emotion: neutral
  Face: ./faces/face_44_0.jpg → Emotion: neutral
  Face: ./faces/face_45_0.jpg → Emotion: neutral
  Face: ./faces/face_46_0.jpg → Emotion: neutral
  Face: ./faces/face_47_0.jpg → Emotion: neutral
  Face: ./faces/face_48_0.jpg → Emotion: neutral
  Face: ./faces/face_51_0.jpg → Emotion: neutral
  Face: ./faces/face_92_0.jpg → Emotion: sad
  Face: ./faces/face_97_0.jpg → Emotion: neutral
  Face: ./faces/face_123_0.jpg → Emotion: neutral
  Face: ./faces/face_124_0.jpg → Emotion: neutral
  Face: ./faces/face_125_0.jpg → Emotion: neutral
  Face: ./faces/face_126_0.jpg → Emotion: neutral
  Face: ./faces/face_127_0.jpg → Emotion: neutral
  Face: ./faces/face_128_0.jpg → Emotion: neutral
  Face: ./faces/face_129_0.jpg → Emotion: neutral
  Face: ./faces/face_130_0.jpg → Emotion: neutral
  Face: ./faces/face_183_0.jpg → Emotion: angry
  Face: ./faces/face_257_0.jpg → Emotion: sad
  Face: ./faces/face_520_0.jpg → Emotion: sad
  Face: ./faces/face_526_0.jpg → Emotion: sad
  Face: ./faces/face_530_0.jpg → Emotion: fear
  Face: ./faces/face_535_0.jpg → Emotion: sad
  Face: ./faces/face_536_0.jpg → Emotion: sad
  Face: ./faces/face_541_0.jpg → Emotion: sad
  Face: ./faces/face_542_0.jpg → Emotion: sad
  Face: ./faces/face_543_0.jpg → Emotion: sad
  Face: ./faces/face_544_0.jpg → Emotion: fear
  Face: ./faces/face_545_0.jpg → Emotion: fear
  Face: ./faces/face_546_0.jpg → Emotion: fear
  Face: ./faces/face_548_0.jpg → Emotion: sad
  Face: ./faces/face_549_0.jpg → Emotion: sad
  Face: ./faces/face_550_0.jpg → Emotion: sad
  Face: ./faces/face_551_0.jpg → Emotion: sad
  Face: ./faces/face_552_0.jpg → Emotion: sad

```

```
Analyzing video: /content/drive/MyDrive/avengers.mp4
INFO:pyscenedetect:Detecting scenes...

Detected 29 scenes:

Scene 1:
Start Time: 0.00s
End Time: 87.10s
Duration: 87.10s

Scene 2:
Start Time: 87.10s
End Time: 118.57s
Duration: 31.47s

Scene 3:
Start Time: 118.57s
End Time: 119.97s
Duration: 1.40s

Scene 4:
Start Time: 119.97s
End Time: 157.50s
Duration: 37.53s
```

Detection scene difference in videos with start time and end time

```
Processing: SpiderMan.mp4
INFO:fer:23.98 fps, 14451 frames, 602.73 seconds
INFO:fer:Making directories at output
INFO:fer:Deleted pre-existing output/SpiderMan_output.mp4
28%|██████      | 4077/14451 [16:44<29:51, 5.79frames/s]
```

Collection of frames to process video associated to emotions



Sad Emotion detected



Happy Emotion detected



In media libraries or video streaming services, scene detection helps index videos into chapters or topics based on scene transitions, making search and retrieval more efficient. Automated Video Editing: For video editors, scene detection can automate tasks such as cutting, trimming, or reordering video scenes based on the content flow. In surveillance videos, PySceneDetect could be useful in detecting significant events by recognizing scene changes that might signify an incident or unusual activity.


```
sad.py X
1 import os
2
3 import cv2
4 from fer import FER
5
6 emotion_detector = FER(mtcnn=True)
7
8 directory = "/home/anirudh/detect-scene/movies-videos"
9 output_dir = "/home/anirudh/detect-scene/movie_output_sad_videos"
10
11 os.makedirs(output_dir, exist_ok=True)
12 #
13 for name in os.listdir(directory):
14     path_to_video = os.path.join(directory, name)
15
16     if not os.path.isfile(path_to_video):
17         continue
18
19     print(f"Processing: {name}")
20
21     # path_to_video = "/home/anirudh/detect-scene/videos/3idiots.mp4"
22     cap = cv2.VideoCapture(path_to_video)
23     fps = int(cap.get(cv2.CAP_PROP_FPS))
24     width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH) * 0.5)
25     height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT) * 0.5)
26
27     output_path = os.path.join(output_dir, name + "_sad_video.mp4")
28
29     fourcc = cv2.VideoWriter_fourcc(*"mp4v")
30     out = cv2.VideoWriter(output_path, fourcc, fps // 3, (width, height))
31
32     frame_count = 0
33     sad_frames = 0
34
35     while cap.isOpened():
36         ret, frame = cap.read()
37         if not ret:
38             break
39
40         if frame_count % 3 == 0:
41             frame = cv2.resize(frame, (width, height))
```



```

output_path = os.path.join(output_dir, name + "_sad_video.mp4")

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps // 3, (width, height))

frame_count = 0
sad_frames = 0

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    if frame_count % 3 == 0:
        frame = cv2.resize(frame, (width, height))
        result = emotion_detector.detect_emotions(frame)

        for face in result:
            emotions = face["emotions"]
            dominant_emotion = max(emotions, key=emotions.get)

            # if emotions["sad"] >= 0.40 or dominant_emotion == "sad":
            if emotions["sad"] >= 0.20:
                box = face["box"]
                cv2.rectangle(frame, (box[0], box[1]), (box[0] + box[2], box[1] + box[3]), (255, 0, 0),
                               2)
                cv2.putText(frame, "SAD", (box[0], box[1] - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 0), 2)
                out.write(frame)
                sad_frames += 1

        frame_count += 1

cap.release()
out.release()

print(f"Processed video saved as {output_path} with {sad_frames} 'sad' frames.")

```

CHAPTER 7

REFLECTION AND ANALYSIS

FER (Facial Expression Recognition):

Facial Expression Recognition (FER) is the task of identifying emotions or expressions from human faces, a crucial component in applications like user sentiment analysis, security surveillance, customer feedback systems, and interactive AI-based assistants. The primary focus is on detecting emotions such as happiness, sadness, anger, surprise, disgust, and fear from images or video frames.

FER models are generally based on deep learning techniques, specifically convolutional neural networks (CNNs). However, their accuracy can drop under challenging conditions like varying lighting, occlusion (e.g., glasses or hair covering part of the face), or unusual facial expressions.

Pre-trained models, such as those from libraries like FER in Python, make the deployment of FER easier, reducing the need for training large models from scratch.

Applications & Use Cases:

Customer Sentiment Analysis: Many businesses use FER to gauge user sentiment by analyzing facial expressions during video calls or in-store interactions.

Human-Computer Interaction: FER is used in gaming, virtual assistants, and robotics to create more human-like interactions by understanding the user's emotional state. **Mental Health and Diagnostics:** FER models are increasingly used in detecting signs of mental health issues, like depression or anxiety, based on facial expressions over time.

Ethical Considerations: FER systems can raise privacy and ethical issues, especially when used in sensitive environments like workplaces or public spaces. There's a risk of misinterpreting or over-interpreting emotions, potentially leading to bias.

Cultural Differences: Emotions might be expressed differently in various cultures, leading to challenges in generalizing FER models across diverse populations.

Analysis of FER:

Strengths: The FER Python library is a powerful tool for quickly detecting and classifying emotions from images and videos. It uses pre-trained models like ResNet and other deep learning architectures for real-time emotion detection, which is highly useful for prototyping and low-latency applications.

Weaknesses: The FER library, while versatile, can sometimes struggle with subtle facial expressions or complex scenarios (e.g., overlapping emotions or non-standard expressions).

Utilized **OpenCV** to process video frame-by-frame and detect hard cuts based on pixel differences. This automated scene segmentation significantly reduced manual editing efforts. Scene segmentation using **OpenCV** and the **FER** library involves detecting and analyzing individual frames from a video or stream to identify emotional expressions within distinct scenes. OpenCV is first used to process the video frame-by-frame, applying face detection algorithms to locate faces within each frame. Once a face is detected, the FER library analyzes the facial features and classifies the emotion, such as happy, sad, angry, or surprised. These emotional cues are then used as a basis for scene segmentation—grouping sequences of frames that share similar emotional content. This allows for the extraction of emotionally coherent scenes, enabling applications such as highlight generation, sentiment-aware video summarization, and emotion-driven content indexing. The combination of OpenCV for visual processing and FER for emotion classification creates a lightweight and effective pipeline for emotion-based scene segmentation.

PySceneDetect:

PySceneDetect is a Python library used for detecting scene boundaries (or cuts) in videos, crucial for video analysis and media processing tasks. It helps in dividing videos into scenes, allowing better indexing, summarization, or further analysis like action recognition, content tagging, or facial expression changes across scenes.

PySceneDetect is a tool for detecting shot changes in videos (example), and can automatically split the video into separate clips. **PySceneDetect** is free and open-source software, and has several detection methods to find fast-cuts and threshold-based fades.

Content-Aware Scene Detection:

Detects breaks in-between content, not only when the video fades to black (although a threshold mode is available as well for those cases).

The detected scene boundaries/cuts can be exported in a variety of formats, with the default type (comma-separated HH:MM:SS.nnn values) being ready to copy-and-paste directly into other tools (such as ffmpeg, mkvmerge, etc...) for splitting and/or re-encoding the video.

Statistical Video Analysis:

Can output a spreadsheet-compatible file for analyzing trends in a particular video file, to determine the optimal threshold values to use with specific scene detection methods/algorithms.

Extendible and Embeddable:

Written in Python, and designed with an easy-to-use and extendable API, PySceneDetect is ideal for embedding into other programs, or to implement custom methods/algorithms of scene detection for specific applications (e.g. analyzing security camera footage).

Features:

- exports timecodes in standard format (HH:MM:SS.nnn), comma-separated for easy copy-and-paste into external tools and analysis with spreadsheet software.
- statistics/analysis mode to export frame-by-frame video metrics via the `-s [FILE]/--stats [FILE]` argument (e.g. `--stats metrics.csv`).
- output-suppression (quiet) mode for better automation with external scripts/programs (`-q/--quiet`).
- save an image of the first and last frame of each detected scene via the `save-images` command.
- split the input video automatically if `ffmpeg` or `mkvmerge` is available via the `split-video` command.

Output Formats:

- EDL: `save-edl` command (save as edit decision list in CMX 3600 format, compatible with most editors)
- HTML: `save-html` command (save HTML table that can be viewed with browser)
- OTIO: `save-otio` command (save as OpenTimelineIO file)
- QP: `save-qp` command (can be used with `x264 --qpfile`)

Detection Methods:

PySceneDetect implements a variety of different detection algorithms which can be used independently or combined depending on the source material being analyzed.

- **Adaptive content scene detection (detect-adaptive):** uses rolling average of differences in HSL colorspace combined with thresholding to detect shot changes (fast cut)
- **Content-aware scene detection (detect-content):** uses differences in HSL colorspace combined with filtering to detect shot changes (fast cut)
- **Content-aware scene detection (detect-hash):** uses perceptual hashing to determine differences between frames to find shot changes (fast cut)

- **Content-aware scene detection (detect-hist):** uses differences in histograms of Y channel of frames after conversion to YUV (fast cut)
- **Threshold scene detection (detect-threshold):** uses average frame intensity (brightness) to detect slow transitions (fade in/out)

Reflection on PySceneDetect:

Accuracy & Performance:

PySceneDetect employs both threshold-based and content-aware (e.g., machine learning or pixel-based) techniques to detect scene transitions, making it suitable for a range of videos from films to home videos.

The content-aware method allows for better accuracy when detecting scene changes where visual content significantly shifts. On the other hand, simple threshold methods might work well in controlled settings (like recorded webinars or lectures) but may be less effective in more dynamic or creative video styles.

Applications & Use Cases:

Content Indexing: In media libraries or video streaming services, scene detection helps index videos into chapters or topics based on scene transitions, making search and retrieval more efficient. **Automated Video Editing:** For video editors, scene detection can automate tasks such as cutting, trimming, or reordering video scenes based on the content flow.

Event Detection: In surveillance videos, PySceneDetect could be useful in detecting significant events by recognizing scene changes that might signify an incident or unusual activity.

Challenges:

Accuracy in Dynamic Content: PySceneDetect can struggle in cases where scene changes are subtle or poorly defined, such as in continuous action scenes or videos with slow transitions.

Video Quality: High compression or low-resolution videos might impact the accuracy of scene detection algorithms, as some scene boundaries might not be as visually distinct.

Analysis of PySceneDetect:

Strengths: PySceneDetect excels at quickly and efficiently segmenting videos into scenes. Its integration with OpenCV makes it a robust tool for video processing workflows. The fact that it provides both simple threshold-based and content-aware detection methods makes it versatile for different types of content.

Weaknesses: Scene detection may not always be perfect, especially in videos with subtle scene changes or continuous, fluid transitions. Depending on the chosen algorithm, the detection might miss some nuanced shifts in content.

Integrating FER and PySceneDetect:

When combining FER and PySceneDetect, you can create a powerful system for analyzing emotional content in videos by detecting when and how emotions evolve across different scenes. Here's how it can be applied:

Real-time Emotion Analysis Across Scenes: For a video, you could apply PySceneDetect to segment it into scenes and then use FER to analyze the emotions of subjects within each scene. This would allow a more granular understanding of how emotions evolve in different parts of a video (e.g., a character's mood changes in different scenes of a movie).

User Sentiment Analysis in Video Content: If analyzing user-generated content, such as customer reviews or tutorials, you could detect shifts in facial expressions (via FER) to understand how the user feels during different parts of the video (e.g., excitement in one scene, frustration in another).

Automated Content Segmentation: When editing videos or creating summaries, FER combined with scene detection could automatically highlight emotional peaks, such as moments of laughter or shock, making content curation more aligned with emotional engagement.

This project pushed the boundaries of traditional scene detection by integrating emotion analysis, adding narrative depth to video segmentation. Implementing PySceneDetect offered a solid foundation for cut-based scene detection. It was reliable, fast, and easy to customize. Using OpenCV gave hands-on experience with video frame manipulation, a critical skill in computer vision workflows. FER library integration was particularly rewarding—it opened up emotional dimensions to otherwise visual-only analysis.

Balancing performance with accuracy was a consistent challenge, especially when handling long or high-resolution videos. Timestamp extraction streamlined editing workflows and made results directly usable in media processing pipelines. Debugging frame-by-frame transitions built resilience and deepened understanding of real-world computer vision bottlenecks. The modularity of the system allowed easy testing of different thresholds and configurations. Exporting structured outputs made integration with other tools and platforms effortless.

When combining FER and PySceneDetect, you can create a powerful system for analyzing emotional content in videos by detecting when and how emotions evolve across different scenes. Here's how it can be applied:

Real-time Emotion Analysis Across Scenes: For a video, you could apply PySceneDetect to segment it into scenes and then use FER to analyze the emotions of subjects within each scene. This would allow a more granular understanding of how emotions evolve in different parts of a video (e.g., a character's mood changes in different scenes of a movie).

User Sentiment Analysis in Video Content: If analyzing user-generated content, such as customer reviews or tutorials, you could detect shifts in facial expressions (via FER) to understand how the user feels during different parts of the video (e.g., excitement in one scene, frustration in another).

Video frame indexing and searching: Locates specific frames inside a video by taking keywords or queries/questions in a natural language. For example, a query input such as show me all films where there's a scene for people watching TV traditionally requires the slicing and storage of all video frames in a database. However, this is inefficient when working with multiple videos with a long video length. The scene detection and cut function could store, and process only the representative frames which reduces the searching space. Based on the example in this article, the total number of frames can reduce from 1440 to 44 indicating a 97% reduction in the processing time.

Emotion detection performance varied with lighting and face visibility, which highlighted the importance of preprocessing. Combining multiple Python libraries and ensuring compatibility was a practical software engineering lesson. The dual-purpose approach (scene + emotion) aligned well with content creators' and media analysts' needs. The experience reinforced the importance of balancing creativity with technical feasibility in AI projects. Achieving accurate and reproducible results taught the value of controlled experimentation and threshold tuning.

This project encouraged thinking beyond functionality, focusing on user experience and downstream applications. It also sparked ideas for future extensions like emotion-based video summarization or audio-visual analysis.

Overall, this was a comprehensive, rewarding learning journey across computer vision, emotion AI, and data engineering.

CHAPTER 8

RECOMMENDATIONS

1. Introduce Well-Defined Internship Structures

Entertainment Technologists can enhance future internship experiences by adopting a structured framework that sets clear expectations from the start. Defining roles, goals, timelines, and deliverables will help interns stay focused and aligned. A balanced schedule with interactive sessions and task-based learning can guide skill growth while enabling interns to track their development more effectively.

2. Foster Academic Collaborations

Establishing partnerships with universities and institutions can enrich internships by connecting theory with practical applications. Collaborations through workshops, guest lectures, or joint research activities can introduce interns to real-world challenges in cybersecurity, risk management, and compliance, making their learning more well-rounded.

3. Establish Strong Mentorship Channels

Assigning each intern a mentor who offers technical guidance and regular feedback can make the learning curve smoother. Structured mentor–intern check-ins, feedback reviews, and support can help interns better understand workflows and boost their confidence in applying tools and solving problems independently.

4. Encourage Active Participation and Creative Problem Solving

Creating opportunities for interns to contribute ideas, participate in meetings, and take ownership of smaller projects encourages innovation. This involvement not only enhances their learning but also brings fresh ideas to ongoing work.

5. Create Networking and Exposure Opportunities

Facilitating cross-functional interactions can help interns explore various career paths. Organizing Q&A sessions, informal chats with professionals, or internal demo days allows interns to learn beyond their immediate teams, increasing awareness of the broader tech and cybersecurity landscape.

CHAPTER 9

CONCLUSION

My internship journey at **Entertainment Technologists** has been an incredibly enriching and transformative experience. Over the course of this internship, I not only gained technical knowledge but also developed a deeper understanding of how real-world projects work in a professional environment. My internship at Entertainment Technologists provided an incredible opportunity to apply AI and computer vision skills in a media-focused environment. I developed a comprehensive pipeline that combined scene detection and emotion analysis, delivering accurate and insightful video segmentation. Through hands-on work with tools like **OpenCV**, **FER**, and **PySceneDetect**, I enhanced my technical proficiency and gained real-world experience in handling diverse video data. The project not only improved my coding and debugging skills but also taught me the importance of scalability, performance optimization, and clean data structuring.

This internship significantly strengthened my foundation in AI-driven media solutions and has prepared me for more advanced challenges in the domain of intelligent video analysis. Moreover, I got to see how mentorship and guidance from experienced team members can make a huge difference. Regular feedback and review sessions pushed me to think critically and improve with every task. The internship not only gave me technical confidence but also helped me understand the importance of communication, problem-solving, and team spirit in a work setting.

In conclusion, my time at **Entertainment Technologists** has been a valuable steppingstone in my career journey. It has boosted my confidence as a developer, given me practical skills, and taught me how to adapt, learn, and contribute in a fast-paced tech environment. I am truly grateful for this experience and excited to apply what I've learned to future opportunities.

CHAPTER 10

REFERENCES

- [1] McCarthy, John. What is artificial intelligence? URL: <http://www.formal.stanford.edu/jmci/whatisai.html>, 1998.
- [2] L. Steels, The artificial life roots of artificial intelligence, Artificial Life. 1(1994). 75-110.
- [3] I. Mpiperis, S. Malassiotis, V. Petridis and M. G. Strintzis, 3D facial expression recognition using swarm intelligence, 2008 IEEE International Conference on Acoustics, Speech and Signal Processing, Las Vegas, NV. (2008). 2133-2136 .10.1109/ICASSP.2008.4518064.
- [4] C. Mayer, M. Wimmer, M. Eggers and B. Radig, Facial Expression Recognition with 3D Deformable Models, 2009 Second International Conferences on Advances in Computer-Human Interactions, Cancun. (2009). 26-31.10.1109/ACHI.2009.33
- [5] D. Datcu and L. J. M. Rothkrantz, Automatic recognition of facial expressions using Bayesian belief networks, 2004 IEEE International Conference on Systems, Man and Cybernetics . The Hague 3(2004). 2209-2214. : 10.1109/ICSMC.2004.1400656
- [6] Q. Zhen, D. Huang, Y. Wang and L. Chen, Muscular Movement Model-Based Automatic 3D/4D Facial Expression Recognition, in IEEE Transactions on Multimedia, 18(2016) .1438-1450. 10.1109/TMM.2016.2557063
- [7] V. Gomathi, K. Ramar, A. S. Jeevakumar, Human Facial Expression Recognition using MANFIS Model, International Journal of Computer Science and Engineering, 3(2009) .93-97.
- [8] A. Ramirez Rivera, J. A. Rojas Castillo and O. Chae, Recognition of face expressions using Local Principal Texture Pattern, 2012 19th IEEE International Conference on Image Processing, Orlando, FL. (2012). 2609-2612.10.1109/ICIP.2012.6467433.